

Computer Vision and Image Processing Project (2025/26)

Deep Learning for Traffic Sign Recognition using the GTSRB Dataset

Ana Oliveira
MEI, PG61510

André Miranda
MEI, PG60238

Maria Matos
MEI, PG61533

Abstract—This report presents a deep learning project focused on traffic sign recognition using the German Traffic Sign Recognition Benchmark (GTSRB) dataset. The main objective was to study the impact of different data augmentation techniques, image preprocessing strategies and ensemble learning techniques on the performance of convolutional neural networks (CNNs). Several augmentation configurations were evaluated, including geometric transformations, color-based augmentations, combined augmentations, massive dataset expansion and MixUp, followed by the construction of ensembles using Soft Voting and Hard Voting methods. Experimental results demonstrate that data augmentation significantly improves model generalization and robustness, while ensemble learning enhances... **TO FINISH**

Index Terms—Deep Learning, Computer Vision, Traffic Sign Recognition, GTSRB, Convolutional Neural Networks, Data Augmentation, Image Processing, Ensemble Learning

I. INTRODUCTION

Traffic sign recognition is an important task in computer vision and autonomous driving systems. Modern Advanced Driver Assistance Systems (ADAS) rely heavily on robust visual recognition models capable of detecting and classifying traffic signs under different environmental conditions such as illumination changes, rotations, occlusions, and motion blur.

The objective of this project is to explore deep learning techniques applied to the GTSRB dataset in order to achieve the highest possible classification accuracy. According to published benchmarks, the best reported accuracy on the dataset is approximately 99.82%.

In the first phase, several image preprocessing and data augmentation techniques are evaluated. The goal is to understand how different augmentation strategies influence model generalization and robustness.

In the second phase, ensemble learning techniques are explored. Multiple models trained during Phase 1 are combined using voting strategies in order to reduce prediction variance and improve overall classification performance.

II. DATASET

The German Traffic Sign Recognition Benchmark (GTSRB) is a well-known benchmark dataset for traffic sign classification.

The dataset contains:

- 43 traffic sign classes;
- More than 50,000 images;

- Images captured under real-world conditions;
- Significant variability in illumination, perspective, scale, and occlusion.

The dataset was divided into:

- Training set;
- Validation set;
- Test set.

A. Class Distribution

One important characteristic of the dataset is the imbalance between classes. Some traffic signs contain considerably more samples than others, which can introduce bias during training.

The initial dataset analysis revealed:

- Strong variability in the number of samples per class;
- Variability in image resolution;
- Presence of noisy samples and illumination changes.

III. METHODOLOGY

A. Framework and Libraries

The project was implemented using:

- Python;
- PyTorch;
- Torchvision;
- NumPy;
- Matplotlib;
- Pandas.

B. Image Preprocessing

All images were resized to a fixed spatial resolution before being provided to the neural networks. Resizing ensures consistent tensor dimensions and allows efficient mini-batch processing during training.

The preprocessing pipeline included:

- Image resizing;
- Conversion to tensor format;
- Floating-point normalization;
- Standardization when required by pretrained models.

Example preprocessing pipeline:

```
transform_base = v2.Compose([
    v2.Resize((IMG_SIZE, IMG_SIZE)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True)
])
```

For transfer-learning experiments using VGG16, ImageNet normalization statistics were applied to match the distribution of the pretrained weights.

IV. MODEL ARCHITECTURE

Three different convolutional architectures were evaluated.

A. Custom CNN (Conv_II)

A custom convolutional neural network was developed specifically for the GTSRB dataset. The architecture includes:

- Multiple convolutional layers;
- Batch normalization;
- ReLU activations;
- Max-pooling operations;
- Fully connected classification layers;
- Dropout regularization.

This architecture was designed to balance computational efficiency and classification accuracy.

B. VGG16

The second architecture evaluated was VGG16. In this configuration, the network was initialized without pretrained weights and trained directly on the GTSRB dataset. The original classifier was replaced with a custom classification head composed of a fully connected layer with 256 neurons, a ReLU activation function, a dropout layer with a rate of 0.5, and a final output layer corresponding to the 43 traffic sign classes.

For this architecture, images were resized to a dedicated input resolution and normalized before being fed to the network. The objective was to evaluate the performance of a deeper architecture when trained exclusively on the available traffic sign data.

C. VGG16 with Transfer Learning

A second VGG16-based approach was implemented using transfer learning. In this configuration, the network was initialized with pretrained weights, and all feature extraction layers were frozen during training. The original classifier was replaced by the same custom classification head used in the previous experiment, consisting of a fully connected layer with 256 neurons, ReLU activation, dropout regularization, and a final classification layer with 43 outputs.

Only the classifier layers were updated during training, while the pretrained feature extraction layers remained unchanged. This approach was evaluated to determine whether reusing previously learned visual representations could improve classification performance on the GTSRB dataset.

D. Training Strategy

All experiments used the same training configuration to ensure a fair comparison between augmentation techniques.

The training pipeline included:

- Adam optimizer;
- Cross-entropy loss;
- Early stopping;
- Learning-rate scheduling using ReduceLROnPlateau;

- Model checkpointing based on validation performance.

To reduce the influence of random initialization, each configuration was trained multiple times and the average test accuracy was recorded.

V. PHASE 1 - DATA AUGMENTATION STRATEGIES

The first phase is focused on evaluating the impact of data augmentation techniques on model performance.

Data augmentation is a key technique to improve the generalization ability of deep learning models, especially in image classification tasks such as traffic sign recognition. In this work, multiple augmentation strategies were systematically designed and evaluated using the GTSRB dataset.

The main objective was to analyze how different transformations affect model robustness and accuracy under real-world variations such as rotation, illumination changes, occlusions, and perspective distortion.

All experiments follow a controlled evaluation protocol using the same CNN architecture, optimizer (Adam), and training configuration, varying only the augmentation pipeline.

A. Experiment 0 - Baseline

The baseline experiment uses only minimal preprocessing, without any augmentation applied to the training data. This serves as the reference for all subsequent comparisons.

```
t_base = transform_base_vgg if model_name in ('vgg16',
'vgg16_tl') else transform_base
_, mean, std = run_experiment(
    name='baseline',
    transform_train=t_base,
    transform_test=t_base,
    model_name=model_name
)
```

B. Experiment 1 - Geometric Augmentation

This experiment introduces basic geometric transformations, aiming to improve invariance to small spatial changes such as rotation and translation.

```
transform_geo = v2.Compose([
    v2.RandomRotation(15, interpolation=v2.
InterpolationMode.BILINEAR),
    v2.Resize((IMG_SIZE_CONV, IMG_SIZE_CONV)),
    v2.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True)
])

transform_geo_vgg = v2.Compose([
    v2.RandomRotation(15, interpolation=v2.
InterpolationMode.BILINEAR),
    v2.Resize((IMG_SIZE_VGG, IMG_SIZE_VGG)),
    v2.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize([0.485, 0.456, 0.406],
[0.229, 0.224, 0.225])
])
```

These transformations simulate small camera shifts and orientation changes commonly found in real driving scenarios.

C. Experiment 2 - Advanced Geometric Augmentation

A more aggressive geometric pipeline was introduced using perspective distortion and random cropping. This aims to simulate more extreme viewpoint variations.

```
transform_geo2 = v2.Compose([
    v2.RandomPerspective(distortion_scale=0.2, p=0.6),
    v2.Resize((IMG_SIZE_CONV + 8, IMG_SIZE_CONV + 8)),
    v2.RandomCrop((IMG_SIZE_CONV, IMG_SIZE_CONV)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True)
])

transform_geo2_vgg = v2.Compose([
    v2.RandomPerspective(distortion_scale=0.2, p=0.6),
    v2.Resize((IMG_SIZE_VGG + 8, IMG_SIZE_VGG + 8)),
    v2.RandomCrop((IMG_SIZE_VGG, IMG_SIZE_VGG)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize([0.485, 0.456, 0.406],
                 [0.229, 0.224, 0.225])
])
```

This setup introduces stronger spatial variability while maintaining the semantic structure of traffic signs.

D. Experiment 3 - Color Augmentation

This experiment focuses on photometric transformations using ColorJitter operations. Brightness, contrast, saturation and hue were randomly modified to simulate different illumination and weather conditions.

```
transform_color = v2.Compose([
    v2.Resize((IMG_SIZE_CONV, IMG_SIZE_CONV)),
    v2.ColorJitter(brightness=0.5, contrast=0.5,
                  saturation=0.3, hue=0.05),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True)
])

transform_color_vgg = v2.Compose([
    v2.Resize((IMG_SIZE_VGG, IMG_SIZE_VGG)),
    v2.ColorJitter(brightness=0.5, contrast=0.5,
                  saturation=0.3, hue=0.05),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize([0.485, 0.456, 0.406],
                 [0.229, 0.224, 0.225])
])
```

Color jittering is particularly relevant for real-world driving scenarios where illumination is highly variable.

E. Experiment 4 - Combined Augmentation

This experiment combines geometric and color augmentations, along with random erasing to simulate occlusions such as dirt, shadows, or partial obstruction.

```
transform_combined = v2.Compose([
    v2.RandomRotation(15, interpolation=v2.
        InterpolationMode.BILINEAR),
    v2.Resize((IMG_SIZE_CONV, IMG_SIZE_CONV)),
    v2.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    v2.ColorJitter(brightness=0.5, contrast=0.5,
                  saturation=0.3, hue=0.05),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.RandomErasing(p=0.3, scale=(0.02, 0.1),
                    value='random')
])
```

```
transform_combined_vgg = v2.Compose([
    v2.RandomRotation(15, interpolation=v2.
        InterpolationMode.BILINEAR),
    v2.Resize((IMG_SIZE_VGG, IMG_SIZE_VGG)),
    v2.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    v2.ColorJitter(brightness=0.5, contrast=0.5,
                  saturation=0.3, hue=0.05),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.RandomErasing(p=0.3, scale=(0.02, 0.1),
                    value='random'),
    v2.Normalize([0.485, 0.456, 0.406],
                 [0.229, 0.224, 0.225])
])
```

Random erasing improves robustness by forcing the network to learn partial-feature representations.

F. Experiment 5 - Massive Dataset Expansion

This experiment significantly increases dataset diversity by duplicating the training dataset with a strongly augmented version. The final dataset is a concatenation of original and augmented samples.

```
transform_strong = v2.Compose([
    v2.RandomRotation(20, interpolation=v2.
        InterpolationMode.BILINEAR),
    v2.Resize((IMG_SIZE_CONV, IMG_SIZE_CONV)),
    v2.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    v2.ColorJitter(brightness=0.5, contrast=0.5,
                  saturation=0.3, hue=0.05),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.RandomErasing(p=0.3, scale=(0.02, 0.1),
                    value='random')
])

transform_strong_vgg = v2.Compose([
    v2.RandomRotation(20, interpolation=v2.
        InterpolationMode.BILINEAR),
    v2.Resize((IMG_SIZE_VGG, IMG_SIZE_VGG)),
    v2.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    v2.ColorJitter(brightness=0.5, contrast=0.5,
                  saturation=0.3, hue=0.05),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.RandomErasing(p=0.3, scale=(0.02, 0.1),
                    value='random'),
    v2.Normalize([0.485, 0.456, 0.406],
                 [0.229, 0.224, 0.225])
])
```

This strategy effectively doubles the dataset size, improving generalization capacity.

The model is trained multiple times with different random initializations to ensure statistical reliability:

```
for i in range(RUNS):
    if model_name == 'conv_II':
        model = Conv_II(NUM_CLASSES).to(device)
    elif model_name == 'vgg16':
        model = torchvision.models.vgg16()
    ...
    train(...)
    acc = evaluate(model, test_loader)
```

This setup also includes:

- Learning rate scheduling (ReduceLROnPlateau);
- Early stopping;
- Model checkpointing.

G. Experiment 6 - MixUp and CutMix Augmentation

This experiment evaluates the impact of advanced data augmentation techniques, specifically MixUp and CutMix, on model generalization. Both methods generate synthetic training samples to improve robustness and reduce overfitting.

MixUp creates new training examples by linearly interpolating pairs of images and their corresponding labels, producing soft labels that encourage smoother decision boundaries. Meanwhile, CutMix improves augmentation diversity by replacing a random patch of one image with a patch from another image, while adjusting the labels proportionally to the area of the patch.

```
mixup = v2.MixUp(num_classes=NUM_CLASSES)
history = {'loss': [], 'accuracy': [],
          'val_loss': [], 'val_acc': []}
best_val_loss = np.inf

...

transform_mix = v2.Compose([
    v2.Resize((IMG_SIZE_CONV, IMG_SIZE_CONV)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
])

transform_mix_vgg = v2.Compose([
    v2.Resize((IMG_SIZE_VGG, IMG_SIZE_VGG)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize([0.485, 0.456, 0.406],
                 [0.229, 0.224, 0.225])
])
```

Unlike standard training, MixUp produces soft labels, improving decision boundaries and reducing overfitting. CutMix (when applied) similarly acts as a regularization technique by enforcing the model to learn from partial image regions and mixed labels.

During evaluation, standard validation is performed without MixUp or CutMix to ensure fair performance measurement.

VI. PHASE 1 RESULTS AND DISCUSSION

The experimental results demonstrate that data augmentation has a significant influence on classification performance.

Several observations can be highlighted:

- Baseline models already achieved strong performance due to the relatively large size and quality of the GTSRB dataset.
- Moderate geometric transformations improved robustness to viewpoint changes, although excessively aggressive transformations occasionally degraded performance by producing unrealistic samples.
- Photometric augmentation improved robustness to illumination variations while maintaining stable accuracy.
- Combined augmentation increased training diversity but also introduced additional optimization difficulty.
- Massive dataset expansion consistently produced some of the strongest results, indicating that increasing sample diversity is highly beneficial for traffic sign recognition.
- MixUp and CutMix regularization improved generalization by encouraging smoother class boundaries and reducing overfitting.

Overall, the results confirm that augmentation strategies contribute positively to model robustness and generalization.

VII. PHASE 2 – ENSEMBLE LEARNING

Although individual neural networks achieved strong performance, different models tend to make different classification errors. Ensemble learning exploits this diversity by combining multiple predictors into a single decision system.

The objective of Phase 2 was therefore to evaluate whether combining models trained during Phase 1 could further improve classification accuracy.

A. Ensemble Construction

In the second phase of the project, ensemble learning techniques were explored to evaluate whether combining multiple trained models could improve classification performance.

For each augmentation strategy, several models obtained from independent training runs were grouped into an ensemble. Since each model may learn slightly different decision boundaries due to random initialization and training dynamics, combining their predictions can reduce variance and improve robustness.

Two ensemble approaches were evaluated: Soft Voting and Hard Voting.

B. Soft Voting

In the Soft Voting approach, the output scores produced by each model are combined, and the final prediction is determined by the class receiving the highest overall score. This method takes into account the confidence of each model and is often more effective when the individual classifiers provide reliable probability estimates.

C. Hard Voting

In the Hard Voting approach, each model independently predicts a class label. The final prediction is then obtained through majority voting, where the class receiving the largest number of votes is selected.

Unlike Soft Voting, this method only considers the final decision of each model and ignores the confidence associated with the predictions.

D. Experimental Evaluation

Ensembles were evaluated for every augmentation strategy explored in Phase 1:

- Baseline;
- Geometric;
- Advanced Geometric;
- Color;
- Combined;
- Massive Dataset;
- MixUp.

For each configuration, the following metrics were recorded:

- Best individual model accuracy;
- Mean individual model accuracy;
- Soft Voting accuracy;
- Hard Voting accuracy.

VIII. PHASE 2 RESULTS AND DISCUSSION

TO FINISH

Ensemble learning provides two important advantages.

First, averaging predictions reduces variance and mitigates errors caused by individual model initialization.

Second, combining models trained under different random conditions improves robustness and typically increases generalization performance.

The experimental results indicate that

IX. CONCLUSION

TO Finish

This project explored the application of deep learning techniques to traffic sign recognition using the GTSRB dataset.

In the first phase, several data augmentation strategies were evaluated, including geometric transformations, color augmentation, dataset expansion, and MixUp. The results showed that data augmentation plays an important role in improving model generalization and robustness, with stronger augmentation pipelines generally producing better performance.

In the second phase, ensemble learning techniques were investigated by combining models trained during Phase 1. Both Hard Voting and Soft Voting approaches were evaluated, demonstrating that ensembles can ...

Overall, the experiments confirm that combining effective data augmentation with ensemble learning contributes to a ...

Future work may include the exploration of more advanced architectures, such as Residual Networks (ResNets) and Vision Transformers, as well as more sophisticated ensemble techniques like weighted voting and stacking. Additional improvements could also be achieved through hyperparameter optimization, automated augmentation methods, and the use of larger pretrained models to further close the gap to state-of-the-art performance on the GTSRB benchmark.

REFERENCES

- [1] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, *The German Traffic Sign Recognition Benchmark: A multi-class classification competition*, International Joint Conference on Neural Networks (IJCNN), 2011.
- [2] PyTorch Documentation, <https://pytorch.org/>
- [3] Y. LeCun et al., *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 1998.
- [4] A. Shorten and T. Khoshgoftaar, *A survey on Image Data Augmentation for Deep Learning*, Journal of Big Data, 2019.